

Multiprogramming Operating System Project (Phase-I)

Phase 1: Simulation of a basic operating system that executes one job at a time without multiprogramming. It handles simple instruction execution, memory operations, and basic I/O using service interrupts. Students can implement this project by using C/C++ or java Programming language.

Objectives

The main objectives of this project are:

- To simulate the basic functioning of an operating system
- To understand program loading and execution
- To implement interrupt handling
- To simulate memory operations
- To perform basic input/output operations

INTORODUCTION OF PROJECT

A Multiprogramming Operating System (MOS) allows multiple programs to reside in memory and execute by sharing CPU resources. However, in Phase 1 of the MOS project, only one program is loaded and executed at a time. This version focuses on understanding the basic structure of an operating system, including memory management, instruction execution, and interrupt handling.

The system reads programs from an input file, loads them into memory, executes the instructions sequentially, and produces output in an output file.

Project involving the design and implementation of a multiprogramming operating system (MOS) for a hypothetical computer configuration that can be easily simulated (Shaw and Weiderman, 1971). The purpose is to consolidate and apply, in an almost realistic setting, some of the concepts and techniques discussed in this book. In particular, the MOS designer/implementer must deal directly with problems of input-output, interrupt handling, process synchronization, scheduling, main and auxiliary storage management, process and resource data structures, and systems organization.

We assume that the project will be coded for a large central computer facility (The “host” system) which, on the one hand, does not allow users to tamper with the operating system or the machine resources but on the other hand, does provide a complete set of services, including filing services, debugging aids, and a good higher-level language. The global strategy is to simulate the hypothetical computer on the host and writes the MOS for this simulated machine. The MOS and simulator will consist of approximately 1000 to 1200 cards of program, with most of the code representing the MOS. The project can be completed over a period of about two months by students concurrently taking a normal academic load.

MACHINE SPECIFICATIONS

The MOS computer is described from two points of view: the “virtual” machine seen by the typical user and the “real” machine used by the MOS designer/implementer.

1. The Virtual Machine:

The virtual machine viewed by a normal user is illustrated in *Fig. A-1*. Storage consists of a maximum of 100 words, addressed from 00 to 99; each word is divided into four one-byte units, where a byte may contain any character acceptable by the host machine.

The CPU has four registers:

1. a four-byte general register **R**.
2. a four-byte Instruction register **IR**.
3. a two-byte instruction counter **IC**
4. a one-byte Boolean” toggle **C**, which may contain either “**T**” (true) or “**F**” (false),

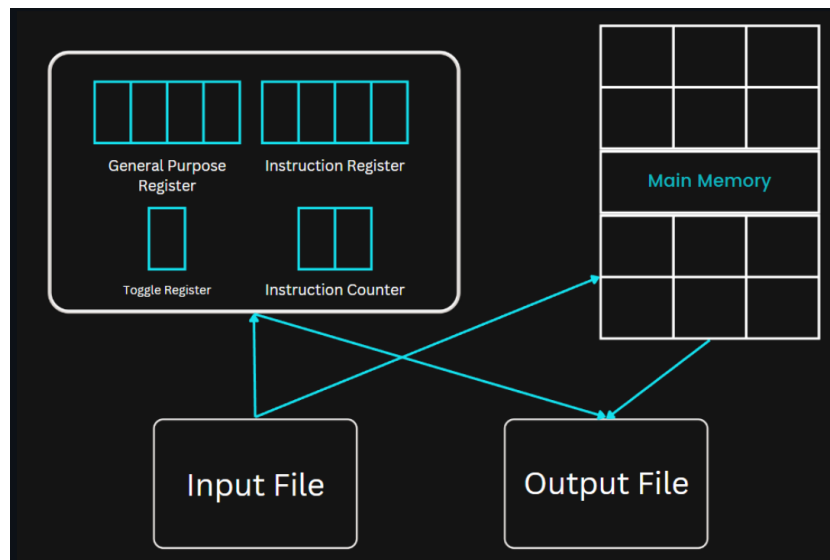


Fig. A-1. Virtual Machine

A storage word may be interpreted as an instruction or data word. The operation code of an instruction occupies the two high-order bytes of the word, and the operand address appears in the two low-order bytes. Note that the input instruction (**GD**) reads only the first 40 bytes of a card (from Input file) and that the output instruction (**PD**) prints a new line of 40 bytes (to output file). The first instruction of a program must *always* appear in location 00. With this simple machine, a batch of compute-bound, IO-bound, and balanced programs can be quickly

written.

2. Registers

Register	Size	Description
IR (Instruction Register)	4 bytes	Stores the currently executing instruction
IC (Instruction Counter)	2 bytes	Points to the next instruction
R (General Purpose Register)	4 bytes	Used for temporary storage
C (Toggle)	1 byte	Condition flag used for comparison

3. Memory

The system contains 100 memory locations, where each location stores one word (4 bytes).

Example:

M[00] = GD10

M[01] = PD10

M[02] = H

4. Instruction Format

Each instruction consists of 4 characters:

Opcode Operand

Example:

GD10

Where:

- GD → Operation code
- 10 → Memory address

5. Instruction Set

Instruction	Description
LR xx	Load register with memory value (works on memory word)
SR xx	Store register value in memory (works on memory word)
CR xx	Compare register with memory (works on memory word)
BT xx	Branch if condition true (works on memory word)
GD xx	Read data into memory (works on Data block)
PD xx	Write data from memory to output (works on memory block)
H	Halt execution

6. Interrupt System

The system uses Service Interrupt (SI) for I/O operations.

SI Value Operation

SI = 1 READ operation

SI = 2 WRITE operation

SI = 3 TERMINATE program

These interrupts are handled by the MOS in Master Mode.

7. Job Control Cards

The input file contains control cards that define program boundaries.

Control Card Meaning

\$AMJ **Start of Job**

\$DTA **Start of Data**

\$END **End of Job**

8. System Assumptions

The following assumptions are made for the MOS implementation:

1. Jobs entered in the input file contain no errors.
2. There is no physical separation between jobs.
3. Job outputs are separated by two blank lines in the output file.
4. Programs are loaded in memory starting from location 00.
5. Only one program runs at a time.
6. A Service Interrupt (SI) is used for service requests.

9. System Modules

The MOS system consists of the following major modules:

1. LOAD
2. STARTEXECUTION
3. EXECUTEUSERPROGRAM
4. MOS (Master Mode Services)

10. Program Execution Steps

A. Program Loading (LOAD)

The LOAD module reads the program from the input file and stores it in memory.

Steps

1. Initialize memory pointer
 $m = 0$

2. Read the next card from input file.
3. If card is \$AMJ
 - Start loading program.
4. If card is a program card
 - Store it in memory locations m to m+9.
 - Increment memory pointer
 $m = m + 10$
5. If card is \$DTA
 - Start program execution.
6. If card is \$END
 - End the job.

B. Start Job Execution

After loading the program, execution begins.

IC = 00

Then the function:

EXECUTEUSERPROGRAM is called.

C. User Program Execution

The EXECUTEUSERPROGRAM module runs instructions sequentially.

Steps

1. Fetch instruction
 $IR = M[IC]$
2. Increment instruction counter
 $IC = IC + 1$
3. Decode operation code
 $IR[1,2]$
4. Execute the corresponding instruction.

D. Instruction Execution

Load Register (LR)

$R \leftarrow M[\text{address}]$

Loads memory value into register.

Store Register (SR)

$M[\text{address}] \leftarrow R$

Stores register value into memory.

Compare Register (CR)

```
If R == M[address]
    C = TRUE
Else
    C = FALSE
```

```
Branch if True (BT)
If C == TRUE
    IC = address
```

```
Get Data (GD)
SI = 1
MOS()
Requests READ operation.
```

```
Put Data (PD)
SI = 2
MOS()
Requests WRITE operation.
```

```
Halt (H)
SI = 3
MOS()
Terminates the program.
```

E. Master Mode (MOS)

The MOS module handles system calls generated by the user program.

Case SI of

- 1 → READ
- 2 → WRITE
- 3 → TERMINATE

F. READ Operation

The READ operation performs the following:

1. Reset operand value
 $IR[4] = 0$
2. Read the next data line from the input file.
3. Store the data into memory from:

$$IR[3,4] \rightarrow IR[3,4] + 9$$

If \$END is encountered, the program aborts due to out-of-data condition.

E. WRITE Operation

The WRITE operation prints data from memory to the output file.

Memory locations

$$IR[3,4] \rightarrow IR[3,4] + 9$$

A block of 10 words is written.

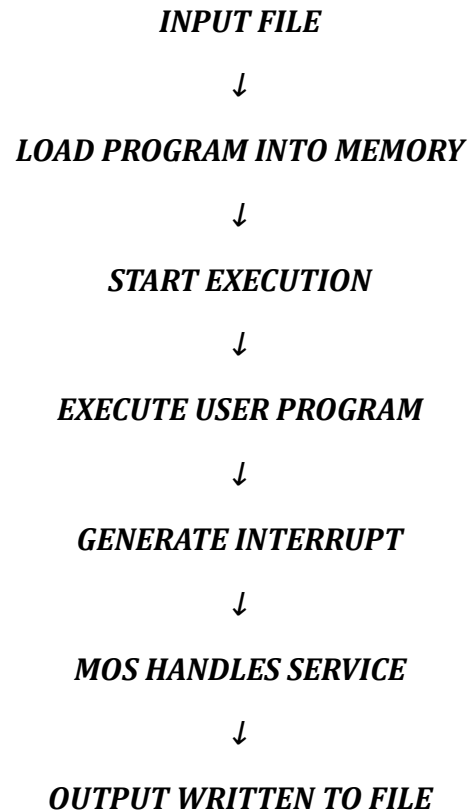
F. TERMINATE Operation

When the program ends:

- Two blank lines are written to the output file.
- Control returns to the operating system.

G. Execution Flow

The overall system execution follows this sequence:



Conclusion

The MOS Phase-1 project demonstrates the basic functioning of an operating system. It simulates how programs are loaded into memory, executed instruction by instruction, and how input/output operations are managed using interrupts.

Although it does not implement true multiprogramming, it provides a foundation for understanding advanced OS concepts such as:

- Process management
- Memory management
- Interrupt handling
- Job control